Java Full Stack Development

2. Normalization

Module Topics

- Normal Forms
- Denormalization

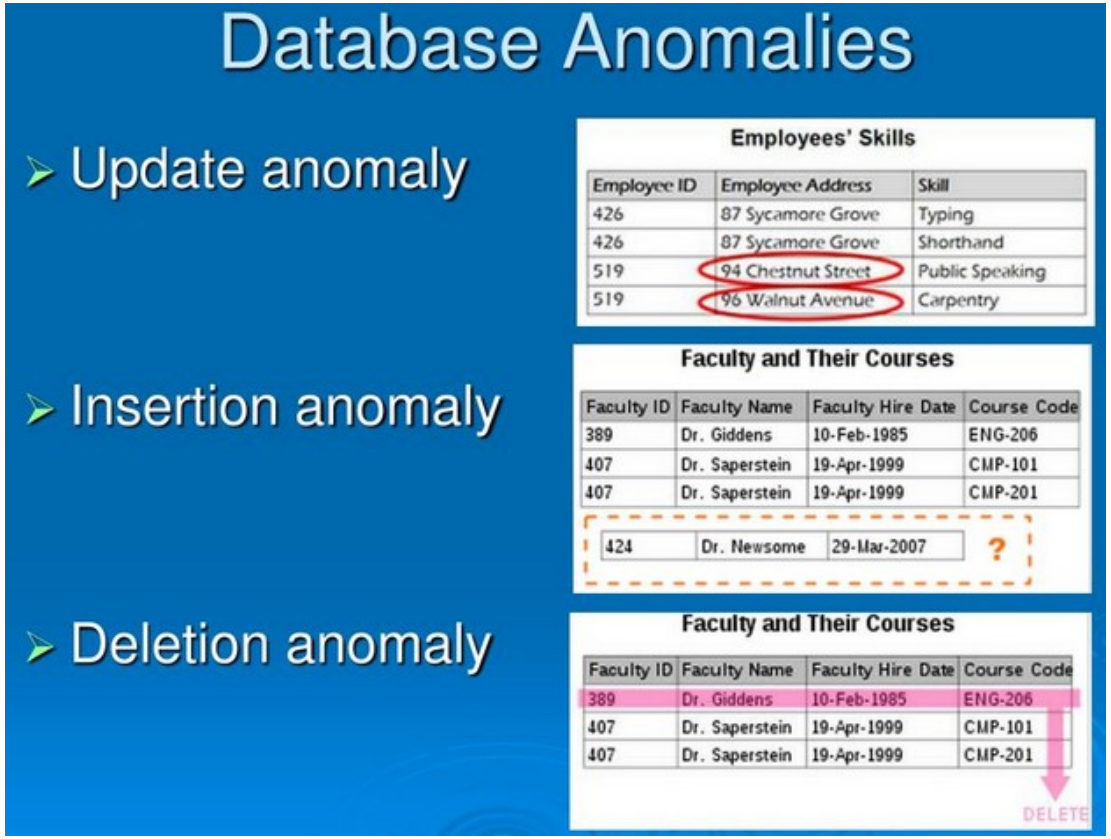
© 2026 by Innovation In Software Corporation

Normal Forms

© 2026 by Innovation In Software Corporation

Anomalies

- The underlying assumption
  - The data in a database accurately represents the real world attributes of the corresponding real world entity
  - For example, if a customer table represents a real life customer
    - We want to be able to assume the customers address is correct
    - And that if update the address, the old address no longer exists
  - In OLTP systems, data is constantly in motion
    - CRUD operations

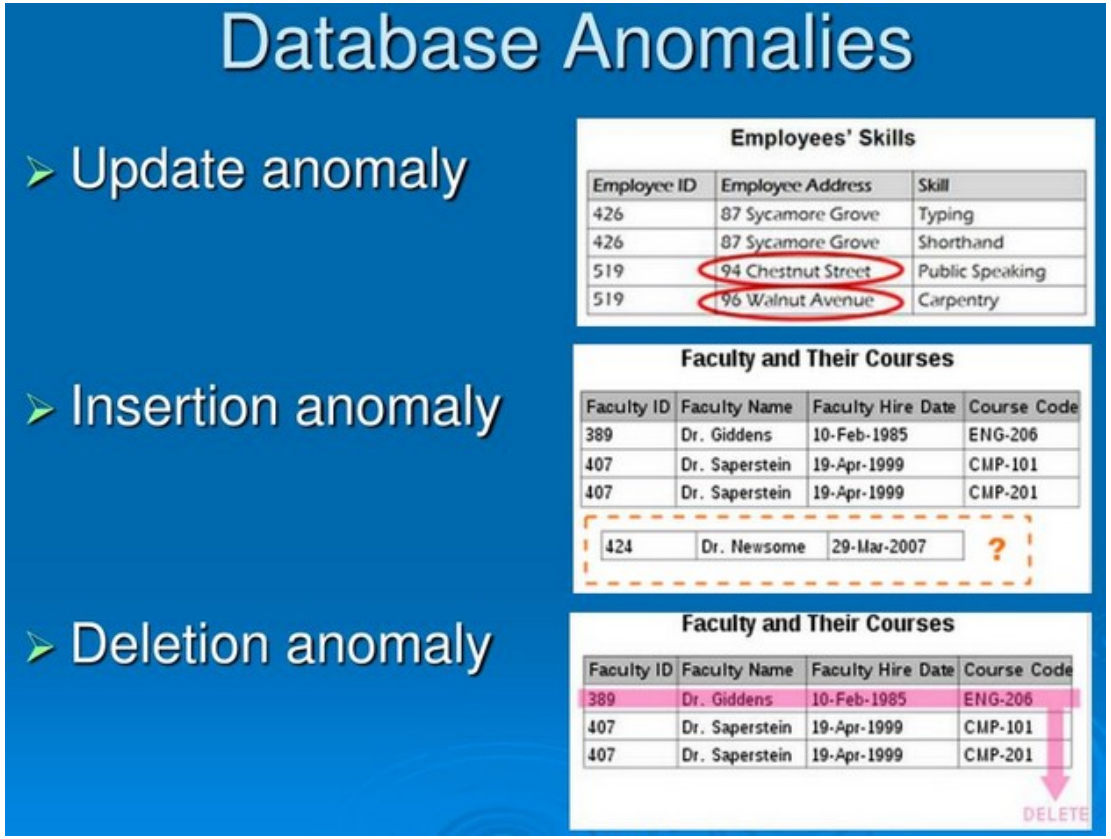


© 2026 by Innovation In Software Corporation

Image Credit: https://www.slideserve.com/niyati/normalization-of-databases

Anomalies

- Every CRUD operation has the potential to corrupt the data
  - These are referred to as anomalies
    - There are no read anomalies because read operations do not alter data
  - Update anomaly
    - Updating attributes creates two copies of data as shown in the image
    - Not clear what the actual address of employee 519 is
    - The underlying cause of the anomaly is that we are storing the address data in two different rows
    - Updating one and not the other creates the anomaly

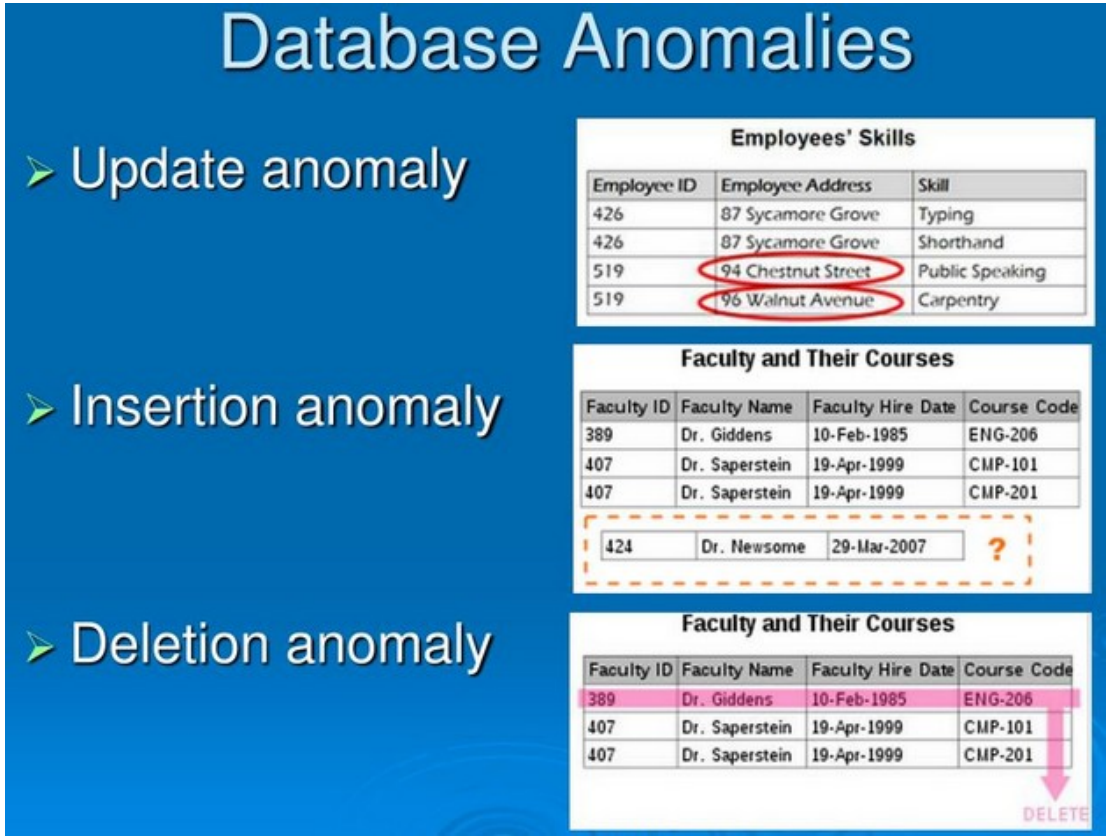


© 2026 by Innovation In Software Corporation

Image Credit: https://www.slideserve.com/niyati/normalization-of-databases

Anomalies

- Every CRUD operation has the potential to corrupt the data
  - Insert anomaly
    - Adding a new record that contains data that is already in the database
    - For example, in the Employee Skill table, adding a new skill for employee 426 requires a new copy of the address that may inconsistent with the existing address
    - In the second table, adding a new professor requires also adding a course code
    - If Dr. Newsome hasn't been assigned a course, we have to add a null for that field

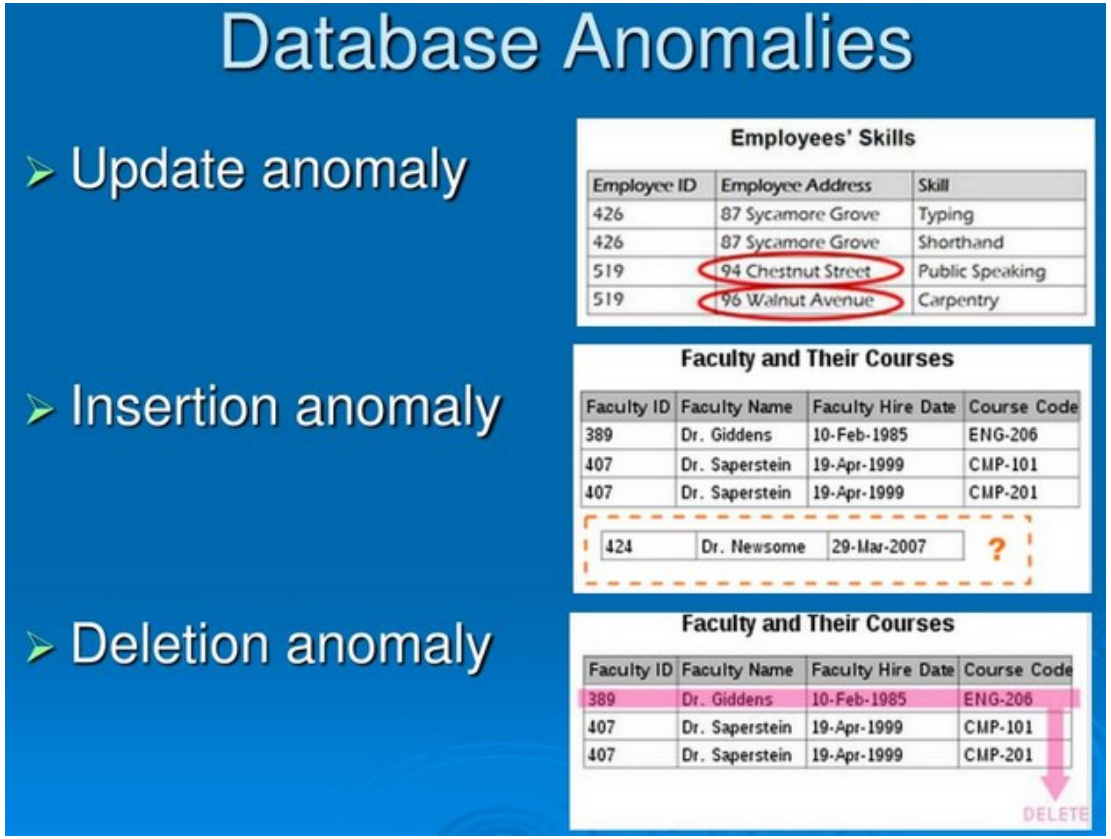


© 2026 by Innovation In Software Corporation

Image Credit: https://www.slideserve.com/niyati/normalization-of-databases

Anomalies

- Every CRUD operation has the potential to corrupt the data
  - Delete anomaly
    - Deleting a record deletes information about more than one entity
    - For example, if employee 426 is the only employee with the typing skill
    - And we delete that record, we also delete the typing skill

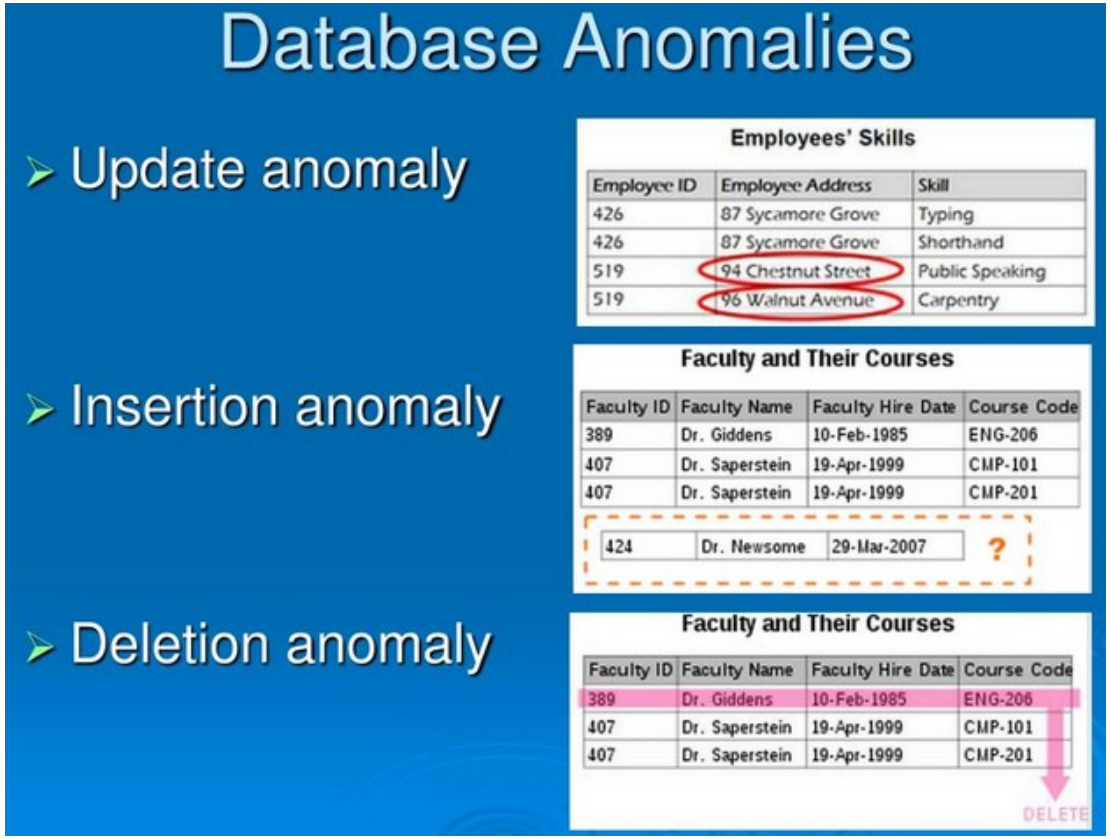


© 2026 by Innovation In Software Corporation

Image Credit: https://www.slideserve.com/niyati/normalization-of-databases

Anomalies

- All anomalies have one of the following related root causes
- A table contains data about more than one entity
  - The examples, the first table contains three entities
    - The employee
    - A skill
    - An address
  - The second table contains two entities
    - The professor
    - The course

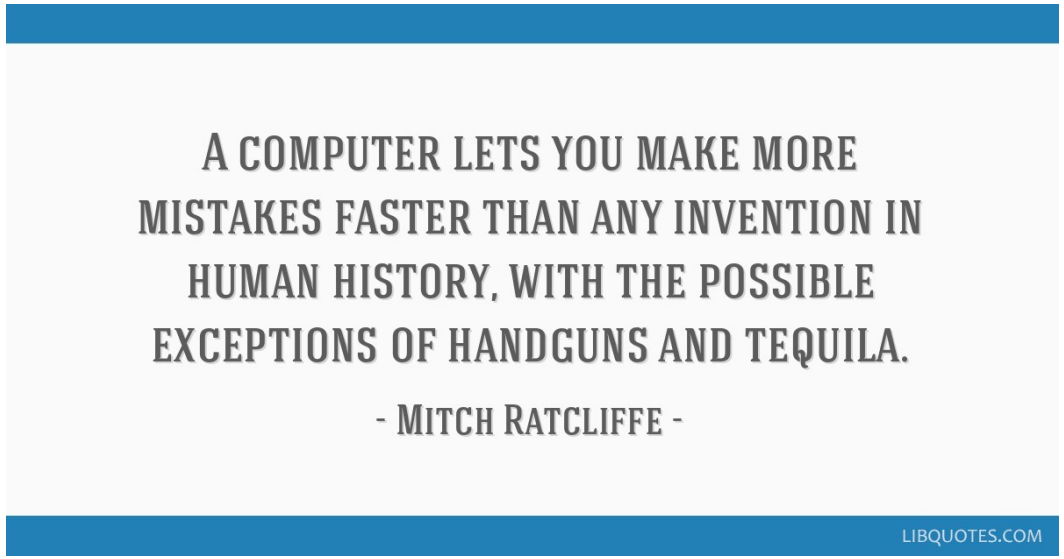


© 2026 by Innovation In Software Corporation

Image Credit: https://www.slideserve.com/niyati/normalization-of-databases

Anomalies

- The same data occurs in more than one place
  - The same address appears in two different records for the same employee



© 2026 by Innovation In Software Corporation

Normal Forms

- The problem normalization solves
  - Without normalization, data is duplicated across rows
    - Duplication leads to update anomalies by updating one copy but not others
    - Duplication leads to insert anomalies, inability to record a fact without recording another
    - Duplication leads to delete anomalies, losing a fact when a related record is deleted
  - Normal forms can be thought of as the implementation of the ideas
    - Every table contains information about only one entity
    - All data items appear only once in the database, there are not duplicates

Form	Rule
1NF	Each column contains atomic values; no repeating groups
2NF	1NF plus every non-key column depends on the whole primary key (relevant for composite keys)
3NF	2NF plus no non-key column depends on another non-key column (no transitive dependencies)
BCNF	A stricter form of 3NF; every determinant is a candidate key
4NF / 5NF	Address multi-valued and join dependencies; rarely applied in practice

© 2026 by Innovation In Software Corporation

Normal Forms

- The normal forms
  - Third Normal Form (3NF) is the standard goal for OLTP schemas
  - Well-normalized schemas are easier to query correctly and easier to maintain
  - If we generate tables directly from a well designed entity-relation logical model
    - It is usually in third normal form automatically
  - Third normal form is often referred to as
    - “All data lookups depend on the key and only the key, so help me Codd”

Form	Rule
1NF	Each column contains atomic values; no repeating groups
2NF	1NF plus every non-key column depends on the whole primary key (relevant for composite keys)
3NF	2NF plus no non-key column depends on another non-key column (no transitive dependencies)
BCNF	A stricter form of 3NF; every determinant is a candidate key
4NF / 5NF	Address multi-valued and join dependencies; rarely applied in practice

© 2026 by Innovation In Software Corporation

Normal Forms

- First normal form (1NF)
  - Intuitively, 1NF says that every column of a table cannot contain a list or multiple entries
  - In the example
    - An un-normalized table is converted into 1NF
    - It still has repeated data because it is not in 2NF
    - This is because the table contains information about two different things, the student and their skills
  - Generally, when a table is put into 1NF, the number of rows increase

Students

FirstName	LastName	Knowledge
Thomas	Mueller	Java, C++, PHP
Ursula	Meier	PHP, Java
Igor	Mueller	C++, Java

Startsituation

Result after Normalisation

Students

FirstName	LastName	Knowledge
Thomas	Mueller	C++
Thomas	Mueller	PHP
Thomas	Mueller	Java
Ursula	Meier	Java
Ursula	Meier	PHP
Igor	Mueller	Java
Igor	Mueller	C++

© 2026 by Innovation In Software Corporation

Image Credit: http://www.gitta.info/LogicModelin/

Normal Forms

- Second normal form (2NF)
  - Specifies that every table contain information about only one entity
  - In the example
    - To list the knowledge areas
    - We have to look up all the students first
  - The 1NF table is still storing information about two different entities
    - To get information about one entity, we have to look up a different entity

Students

FirstName	LastName	Knowledge
Thomas	Mueller	Java, C++, PHP
Ursula	Meier	PHP, Java
Igor	Mueller	C++, Java

Startsituation

Result after Normalisation

Students

FirstName	LastName	Knowledge
Thomas	Mueller	C++
Thomas	Mueller	PHP
Thomas	Mueller	Java
Ursula	Meier	Java
Ursula	Meier	PHP
Igor	Mueller	Java
Igor	Mueller	C++

© 2026 by Innovation In Software Corporation

Image Credit: http://www.gitta.info/LogicModelin/

Normal Forms

- Second normal form
  - Requires that a table already be in 1NF
  - In the example the top table on the left is in 1NF but not 2NF
    - Because each row contains information about three things
  - 2NF splits these into three different tables
    - One for the student, one for the professor and one for the grades
  - Moving to 2NF generally increases the number of tables
    - Most common use of this to create lookup tables in a data base

Students

IDSt	LastName	IDProf	Prof	Grade
1	Mueller	3	Schmid	5
2	Meier	2	Borner	4
3	Tobler	1	Bernasconi	6

Startsituation

Result after normalisation

Students

ID	LastName
1	Mueller
2	Meier
3	Tobler

Professors

IDProf	Professor
1	Bernasconi
2	Borner
3	Schmid

Grades

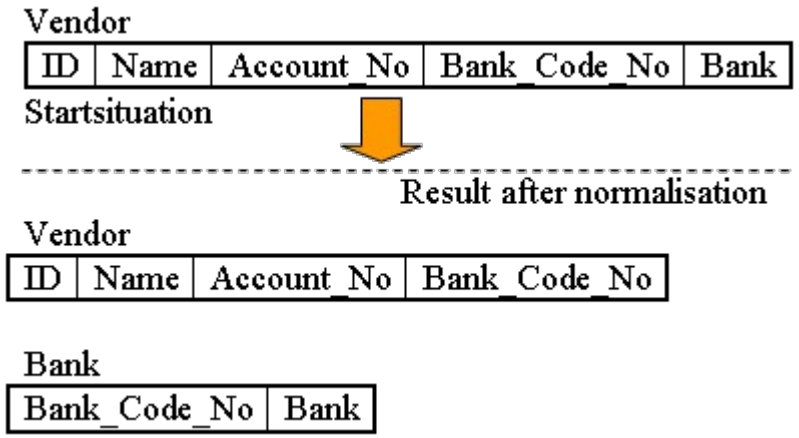
IDStIDProf	Grade	
1	3	5
2	2	4
3	1	6

© 2026 by Innovation In Software Corporation

Image Credit:https://www.slideserve.com/anaya/introduction-to-database-design-methodology

Normal Forms

- Third normal form
  - Requires that a table already be in 2NF
  - And that there be no transitive relations
    - This is more subtle than 2NF
    - Generally occurs when we have an attribute which has multiple parts
  - In the example
    - The vendor has a bank attribute that has two parts: the Bank_Code_No and the Bank (assume this is the name of the bank)
    - The value of the Bank depends on the value in in the Bank_Code_No field
    - We solve this by creating a new table
  - Moving into 3NF can also increases the number of tables



© 2026 by Innovation In Software Corporation

Image Credit:https://www.slideserve.com/anaya/introduction-to-database-design-methodology

Normal Forms

- Fourth normal form
  - Not a common issue
  - Occurs when a row contains two independent facts about an entity
  - In the example, the top table contains two independent facts about a course
    - This implies a relationship between the two independent facts
    - Resolved by separating them out into separate tables

Fourth Normal Form

Course	Instructor	TextBook_Author
Management	X	Churchil
Management	Y	Peters
Management	Z	Peters
Finance	A	Weston
Finance	A	Gilbert

Course

Course	Instructor
Management	X
Management	Y
Management	Z
Finance	A

TextBook Author

Course	Instructor
Management	Churchil
Management	Peters
Finance	Weston
Finance	Gilbert

© 2026 by Innovation In Software Corporation

Image Credit: https://www.geeksforgeeks.org/dbms/introduction-of-4th-and-5th-normal-form-in-dbms/

Normal Forms

- Fourth normal form
  - 4NF violations are less common than 1-3NF
  - They tend to appear in junction tables that were designed to track multiple independent attributes of a relationship simultaneously
  - The signal to look for is a table where removing one row requires removing an unrelated fact at the same time
  - That forced coupling of independent information is usually a 4NF violation

Fourth Normal Form

Course	Instructor	TextBook_Author
Management	X	Churchil
Management	Y	Peters
Management	Z	Peters
Finance	A	Weston
Finance	A	Gilbert

Course

Course	Instructor
Management	X
Management	Y
Management	Z
Finance	A

TextBook Author

Course	Instructor
Management	Churchil
Management	Peters
Finance	Weston
Finance	Gilbert

© 2026 by Innovation In Software Corporation

Image Credit: https://www.geeksforgeeks.org/dbms/introduction-of-4th-and-5th-normal-form-in-dbms/

Normal Forms

- Fifth normal form
  - Very rare
  - It applies when a table can be split into smaller tables and then reconstructed by joining them back together
    - The question is whether we get the original table back after the join
  - In the example, the table on the top has been converted into 4NF in the bottom.

Seller	Company	Product
Rupali	Godrej	Cinthol
Sharda	Dabur	Honey
Sharda	Dabur	HairOil
Sharda	Dabur	Rosewater
Sunil	Amul	Icecream
Sunil	Britania	Biscuits

Seller_Company

Seller	Company
Rupali	Godrej
Sharda	Dabur
Sunil	Amul
Sunil	Britania

Seller_Product

Seller	Product
Rupali	Cinthol
Sharda	Honey
Sharda	HairOil
Sharda	RoseWater
Sunil	Icecream
Sunil	Biscuits

© 2026 by Innovation In Software Corporation

Image Credit: https://cse.poriyaan.in/topic/join-dependencies-and-fifth-normal-form-50871/

Normal Forms

- Fifth normal form
  - However, it is not in 5NF because when we recombine them, we get two rows that were not in the original table

Seller	Company	Product
Rupali	Godrej	Cinthol
Sharda	Dabur	Honey
Sharda	Dabur	HairOil
Sharda	Dabur	Rosewater
Sunil	Amul	Icecream
Sunil	Britania	Biscuits

Seller	Company	Product
Rupali	Godrej	Cinthol
Sharda	Dabur	Honey
Sharda	Dabur	HairOil
Sharda	Dabur	Rosewater
Sunil	Amul	Icecream
Sunil	Amul	Biscuits
Sunil	Britania	Icecream
Sunil	Britania	Biscuits

Newly added records which are not present in original table

© 2026 by Innovation In Software Corporation

Image Credit: https://cse.poriyaan.in/topic/join-dependencies-and-fifth-normal-form-50871/

Normal Forms

- Fifth normal form
  - To put in 5NF, we need to use three tables as shown
- 4NF and 5NF
  - Usually deal with tables that represent multiple relationships between entities
  - Notice in this example
    - The first table tries to capture three different relationships
    - The 5NF and 4NF represent each relationship in its own table

Seller	Company	Product
Rupali	Godrej	Cinthol
Sharda	Dabur	Honey
Sharda	Dabur	HairOil
Sharda	Dabur	Rosewater
Sunil	Amul	Icecream
Sunil	Britania	Biscuits

Seller_Company

Seller	Company
Rupali	Godrej
Sharda	Dabur
Sunil	Amul
Sunil	Britania

Seller_Product

Seller	Product
Rupali	Cinthol
Sharda	Honey
Sharda	HairOil
Sharda	RoseWater
Sunil	Icecream
Sunil	Biscuit

Company_Product

Company	Product
Godrej	Cinthol
Dabur	Honey
Dabur	HairOil
Dabur	RoseWater
Amul	Icecream
Britania	Biscuit

© 2026 by Innovation In Software Corporation

Image Credit: https://cse.poriyaan.in/topic/join-dependencies-and-fifth-normal-form-50871/

Normal Forms

- When working with data
  - 3NF is normally the objective
  - 4NF and 5NF
    - Are quite rare and tend to be associated with complex relationships between entities

Form	Rule
1NF	Each column contains atomic values; no repeating groups
2NF	1NF plus every non-key column depends on the whole primary key (relevant for composite keys)
3NF	2NF plus no non-key column depends on another non-key column (no transitive dependencies)
BCNF	A stricter form of 3NF; every determinant is a candidate key
4NF / 5NF	Address multi-valued and join dependencies; rarely applied in practice

© 2026 by Innovation In Software Corporation

Denormalization

© 2026 by Innovation In Software Corporation

Normal Forms Cost

- The performance trade-off
  - Normalized schemas require joins to reconstruct data
  - Joins are expensive at scale
    - Disk I/O, CPU, memory for hash and sort operations
  - Read-heavy workloads with millions of rows may not tolerate the cost of multi-table joins

Technique	Description	Risk
Storing derived values	Pre-calculating a total or count	Must be kept in sync
Collapsing lookup tables	Storing a status string instead of a FK to a status table	Harder to extend
Duplicating frequently joined columns	Copying a column from a parent table into a child	Update anomalies reintroduced
Materialized views	Oracle-managed pre-computed result sets	Refresh lag; additional storage

© 2026 by Innovation In Software Corporation

Denormalization

- Denormalization is a trade-off, not a shortcut
  - Introduces redundancy in exchange for a measurable performance benefit
  - The risks must be managed
    - Storing a derived total, for example, requires either a trigger or application logic to keep it consistent
  - Materialized views are Oracle's most elegant solution to this problem
    - They pre-compute joins and aggregations and can be refreshed on a schedule or on commit
    - They give most of the performance benefit of denormalization with less risk of inconsistency.

Technique	Description	Risk
Storing derived values	Pre-calculating a total or count	Must be kept in sync
Collapsing lookup tables	Storing a status string instead of a FK to a status table	Harder to extend
Duplicating frequently joined columns	Copying a column from a parent table into a child	Update anomalies reintroduced
Materialized views	Oracle-managed pre-computed result sets	Refresh lag; additional storage

© 2026 by Innovation In Software Corporation

Denormalization

- Legitimate cases for denormalization
  - Reporting and analytics queries that aggregate across large datasets
  - Audit and history tables that must capture a point-in-time snapshot
  - Derived or computed columns cached to avoid repeated calculation
  - Read replicas or materialized views that trade freshness for query speed

Technique	Description	Risk
Storing derived values	Pre-calculating a total or count	Must be kept in sync
Collapsing lookup tables	Storing a status string instead of a FK to a status table	Harder to extend
Duplicating frequently joined columns	Copying a column from a parent table into a child	Update anomalies reintroduced
Materialized views	Oracle-managed pre-computed result sets	Refresh lag; additional storage

© 2026 by Innovation In Software Corporation

Performance Cost

- Normalization splits data across tables
  - Queries must reassemble it
  - A fully normalized schema stores each fact in exactly one place
  - Retrieving a complete picture of any entity requires joining multiple tables
  - Each join is a physical operation
    - Oracle must locate matching rows across two or more separate segments, compare key values, and assemble the result
  - At small data volumes this is fast and the cost is invisible
  - At enterprise scale the cost becomes the dominant factor in query performance

Technique	Description	Risk
Storing derived values	Pre-calculating a total or count	Must be kept in sync
Collapsing lookup tables	Storing a status string instead of a FK to a status table	Harder to extend
Duplicating frequently joined columns	Copying a column from a parent table into a child	Update anomalies reintroduced
Materialized views	Oracle-managed pre-computed result sets	Refresh lag; additional storage

© 2026 by Innovation In Software Corporation

Performance Cost

- The physical cost of a join
  - Each participating table must be accessed
    - Either a full scan of every block in the segment, or an index lookup followed by block reads by ROWID
  - For a hash join
    - Oracle reads one table into memory and builds a hash table
    - Then probes it with rows from the second table
    - Memory demands increases with the number of rows
  - For a nested loop join
    - Oracle reads one row from the outer table and then looks up the matching rows in the inner table
    - Once per outer row: I/O multiplies with the size of the outer result set
  - Every additional join in the query adds another round of I/O and CPU work on top of the previous result

```sql
SELECT p.project_name,
       e.full_name    AS team_lead,
       d.department_name,
       ps.status_label
FROM   projects      p
JOIN   employees     e  ON e.employee_id   = p.lead_employee_id
JOIN   departments   d  ON d.department_id = e.department_id
JOIN   project_status ps ON ps.status_code = p.status
WHERE  p.status = 'ACTIVE';
```

```
projects        -- 500,000 rows, 2,000 blocks
employees       -- 200,000 rows, 800 blocks
departments     -- 5,000 rows,  20 blocks
project_status  -- 10 rows,    1 block  (a lookup table)
```

© 2026 by Innovation In Software Corporation

Performance Cost

- Oracle accesses four separate segments
  - With good indexes, this query is fast for selective predicates
    - But if 400,000 of the 500,000 projects are ACTIVE, the index on status is not selective and Oracle scans the projects segment: 2,000 blocks
  - For each of the 400,000 active project rows, Oracle resolves the employee join, the department join, and the status join
  - Even with perfect indexes, the cumulative I/O across four tables for 400,000 rows is substantial

```
SELECT p.project_name,
       e.full_name    AS team_lead,
       d.department_name,
       ps.status_label
FROM   projects      p
JOIN   employees     e  ON e.employee_id   = p.lead_employee_id
JOIN   departments   d  ON d.department_id = e.department_id
JOIN   project_status ps ON ps.status_code = p.status
WHERE  p.status = 'ACTIVE';
```

```
projects        -- 500,000 rows, 2,000 blocks
employees       -- 200,000 rows, 800 blocks
departments     -- 5,000 rows,  20 blocks
project_status  -- 10 rows,    1 block  (a lookup table)
```

© 2026 by Innovation In Software Corporation

Performance Cost

- Join cardinality multiplication
  - Each join multiplies the number of rows that must be processed
  - A query joining three tables where each join doubles the intermediate result set processes eight times as many rows as a single table scan
  - The optimizer must estimate this multiplication correctly to choose the right join algorithm and order
    - When it gets this wrong, the plan is catastrophically slow
    - Wide queries with five or more joins give the optimizer more opportunities to make a wrong cardinality estimate

```sql
SELECT p.project_name,
       e.full_name    AS team_lead,
       d.department_name,
       ps.status_label
FROM   projects      p
JOIN   employees     e  ON e.employee_id   = p.lead_employee_id
JOIN   departments   d  ON d.department_id = e.department_id
JOIN   project_status ps ON ps.status_code = p.status
WHERE  p.status = 'ACTIVE';
```

```
projects        -- 500,000 rows, 2,000 blocks
employees       -- 200,000 rows, 800 blocks
departments     -- 5,000 rows,  20 blocks
project_status  -- 10 rows,    1 block  (a lookup table)
```

© 2026 by Innovation In Software Corporation

Performance Cost

- Random I/O from index-to-table lookups
  - Joining on a foreign key
    - Typically uses an index range scan on the child table
    - Followed by table block reads via ROWID to retrieve non-indexed columns
  - If the rows returned by the index scan are physically scattered across many blocks in the heap
    - Called “high clustering factor”
    - Each row requires a separate block read
  - A join that returns 50,000 rows from a table with a high clustering factor may read 50,000 different blocks
    - 50,000 random I/Os versus the few hundred blocks a sequential scan would read
  - Common cause for of a query that uses an index but is actually slower than a full table scan

```sql
SELECT p.project_name,
       e.full_name    AS team_lead,
       d.department_name,
       ps.status_label
FROM   projects      p
JOIN   employees     e  ON e.employee_id   = p.lead_employee_id
JOIN   departments   d  ON d.department_id = e.department_id
JOIN   project_status ps ON ps.status_code = p.status
WHERE  p.status = 'ACTIVE';
```

```
projects        -- 500,000 rows, 2,000 blocks
employees       -- 200,000 rows, 800 blocks
departments     -- 5,000 rows,  20 blocks
project_status  -- 10 rows,    1 block  (a lookup table)
```

© 2026 by Innovation In Software Corporation

Denormalization in Practice

- Start normalized
  - Denormalize with evidence.
  - Design the schema in 3NF first
  - Measure actual query performance against realistic data volumes
  - Identify specific queries that exceed latency targets
  - Apply the minimum denormalization that resolves the problem
- Questions to ask before denormalizing
  - Is there an index that would solve the problem without denormalization?
  - Would a materialized view provide the benefit without introducing write-side complexity?
  - Is the query run frequently enough to justify the ongoing maintenance cost?
  - Who is responsible for keeping the redundant data consistent?

© 2026 by Innovation In Software Corporation

Denormalization in Practice

- Where denormalization is almost always wrong
  - OLTP tables with high write concurrency because update anomalies become race conditions
  - Data that changes frequently because synchronization becomes a constant problem
  - Cases where an index or query rewrite would achieve the same result
- Where it is often appropriate
  - Dedicated reporting schemas (data warehouse patterns)
  - Immutable audit or event log tables
  - Materialized summary tables refreshed on a defined schedule

© 2026 by Innovation In Software Corporation

Questions?